

Action Chunking with Transformers (ACT)

NECKER

September 6, 2026

Contents

1	Introduction: Comparing Paradigms (VLA vs ACT)	2
1.1	Architectural Differences and Trade-Offs	2
1.2	Physical Problems Solved by ACT	2
2	The ACT Architecture: End-to-End Tensor Flow	3
2.1	Phase 1: Sensory Ingestion and Visual Abstraction (Encoder)	3
2.1.1	A. The Visual Backbone (ResNet-18) and Feature Maps	3
2.1.2	B. Flattening and 2D Positional Injection	3
2.1.3	C. Proprioception	3
2.2	Phase 2: CVAE (Training Only)	3
2.3	Variational Foundations in ACT: Latent Space, Gaussian Dynamics, and Resolving Multimodality	5
2.3.1	Data Structure: Temporal Chunking and Local Distributions	5
2.3.2	The Latent Variable $\mathbf{z}$ and the Necessity of Variance	6
2.3.3	Objective Function: Balancing Reconstruction and KL Divergence	6
2.3.4	Structural Decoupling Between Training and Inference	7
2.4	Phase 3: The Generative Engine (Transformer Encoder-Decoder)	7
2.4.1	The Transformer Encoder (Implicit Spatial Memory Construction)	7
2.4.2	The Transformer Decoder (Parallel Trajectory Decoding)	8
3	Action Chunking and Temporal Ensembling	14
3.1	Action Chunking (Mitigating Temporal Myopia)	14
3.2	Temporal Ensembling (Exponential Blending)	14
4	Training: Loss Formulation and Gradient Dynamics	15
4.1	Loss Function Components	15
4.2	The Causal Paradox: Why Discard the CVAE at Runtime?	16

1 Introduction: Comparing Paradigms (VLA vs ACT)

In the landscape of Embodied AI, research has developed two profoundly different architectural philosophies to tackle the challenge of robotic control: **Vision-Language-Action (VLA)** models (such as RT-2 or OpenVLA) and purely sensorimotor models such as **ACT (Action Chunking with Transformers)**.

If VLAs simulate the robot’s "prefrontal cortex" (favoring abstract semantic reasoning), ACT embodies its "cerebellum", focusing exclusively on physical dexterity, high-frequency reflexes, and sub-millimeter manipulation.

1.1 Architectural Differences and Trade-Offs

VLA models utilize a Large Language Model (LLM) as their decision-making engine: they fuse visual perception and language into a single latent space, forcing the robot to "think in words" before taking action. In contrast, ACT completely discards the LLM. It relies on a lightweight architecture (a few million parameters versus the billions found in VLAs) based on a **Conditional Variational Autoencoder (CVAE)** and a standard Transformer Encoder-Decoder, which directly maps pixels and proprioception into continuous kinematic coordinates.

This conceptual decoupling produces complementary advantages and drawbacks:

Where VLA models excel (ACT’s limitations):

- **Zero-Shot Generalization and Common Sense:** VLAs inherit the collective knowledge of the internet. They understand the affordances of an object never seen before and can execute logically complex tasks ("Move the apple away from the edge of the table"). ACT is semantically "blind": it does not know what an apple is. It solely learns tight geometric correlations from teleoperation data. When exposed to an out-of-distribution (OOD) object not present in the training set, ACT fails catastrophically.
- **Emergent Reasoning:** VLAs can decompose complex instructions into sequential logical steps and adapt to contextual shifts, whereas ACT mechanically executes the motor pattern it memorized during training.

Where ACT dominates (VLA limitations):

- **Fine Dexterity and Bimanual Control:** VLAs struggle with high-precision tasks (e.g., inserting a USB cable, tying shoelaces, bimanually manipulating fragile items) due to slow inference latency and action tokenization discretization. ACT serves as the *gold standard* for micro-manipulation: it generates continuous coordinate matrices (not discrete tokens) at extremely high control frequencies.
- **Fluidity and Dynamic Stability:** Single-step VLA models suffer from *Compounding Error* (the drift of an individual jerky prediction rapidly derails the overall trajectory). ACT solves this issue by predicting entire blocks of future trajectories (*Action Chunking*) and blending them across time, ensuring physically smooth and safe movements.

1.2 Physical Problems Solved by ACT

Within pure Imitation Learning (*Behavior Cloning*), ACT was deliberately designed to resolve the single greatest obstacle in cloning human behavior: **Multimodality**.

Human demonstrators are inherently inconsistent. Faced with an obstacle, a human may decide to bypass it either from the left or from the right. In the dataset, we therefore have the exact same visual observation $\mathbf{o}_t$ paired with two opposing actions ($\mathbf{a}_{\text{right}}$ and $\mathbf{a}_{\text{left}}$). Minimizing classical loss functions forces a deterministic neural network (such as early BC approaches) to predict the arithmetic mean of both trajectories, leading the robot to crash straight into the obstacle.

ACT bypasses this limitation by leveraging the **CVAE** to encode these behavioral variations within a probabilistic space, enabling the robot to sample a consistent "style" of motion without collapsing into catastrophic averages.

WARNING: Unlike VLA models, ACT is a purely geometric/motor network. It does not reason through natural language. Its objective is not to "understand" the world, but to translate an image and motor joint states into a fluid trajectory that faithfully mirrors the physical dexterity of its human operator.

2 The ACT Architecture: End-to-End Tensor Flow

To fully grasp the mechanics of **Action Chunking with Transformers (ACT)**, one must abandon the Large Language Model paradigm (based on word sequences) and reason strictly in terms of **geometry and spatial perception**.

The ACT model maps pixel arrays and joint angles directly into a continuous trajectory field, utilizing a CVAE to resolve decision bifurcations (e.g., bypassing an obstacle from the left vs. right). Let us trace the flow of tensors from physical sensors through to action prediction.

2.1 Phase 1: Sensory Ingestion and Visual Abstraction (Encoder)

The robot’s instantaneous input $\mathbf{o}_t$ is heterogeneous: it is composed of multi-camera video streams and internal mechanical state vectors.

2.1.1 A. The Visual Backbone (ResNet-18) and Feature Maps

Unlike VLAs that employ massive Vision Transformers (e.g., SigLIP) to extract semantics, ACT prioritizes execution speed and spatial localization. Incoming camera frames (e.g., $I_{\text{top}}, I_{\text{wrist_left}}, I_{\text{wrist_right}} \in \mathbb{R}^{3 \times 480 \times 640}$) are processed by a **standard CNN (ResNet-18)** stripped of its final classification layers.

The ResNet output for each camera is not a single vector, but a **2D Spatial Feature Map**:

$$\mathbf{F}_{\text{cam}} \in \mathbb{R}^{C \times H' \times W'} \quad (\text{e.g., } 512 \times 15 \times 20)$$

This three-dimensional grid preserves scene topology: an activation at the top-left of the map corresponds directly to an object situated at the top-left of the workspace table.

2.1.2 B. Flattening and 2D Positional Injection

Because the downstream Transformer only accepts 1D sequences, the feature maps are "flattened" by collapsing height and width into a single sequence of length $H' \cdot W'$ (e.g., 300 visual tokens per camera).

To preserve geometric information, a **2D Positional Embedding** (bidimensional sinusoidal functions) is added to each vector, informing the Transformer of the original (x, y) coordinates of the abstract pixel patch. Finally, all visual tokens across all cameras are concatenated into a single continuous sequence.

2.1.3 C. Proprioception

In parallel, the joint state $\mathbf{q}_t \in \mathbb{R}^{14}$ (for two 7-DoF robotic arms) is projected by a small MLP into a single embedding vector $\mathbf{e}_{\text{proprio}} \in \mathbb{R}^{d_{\text{model}}}$. This vector is prepended to the visual sequence.

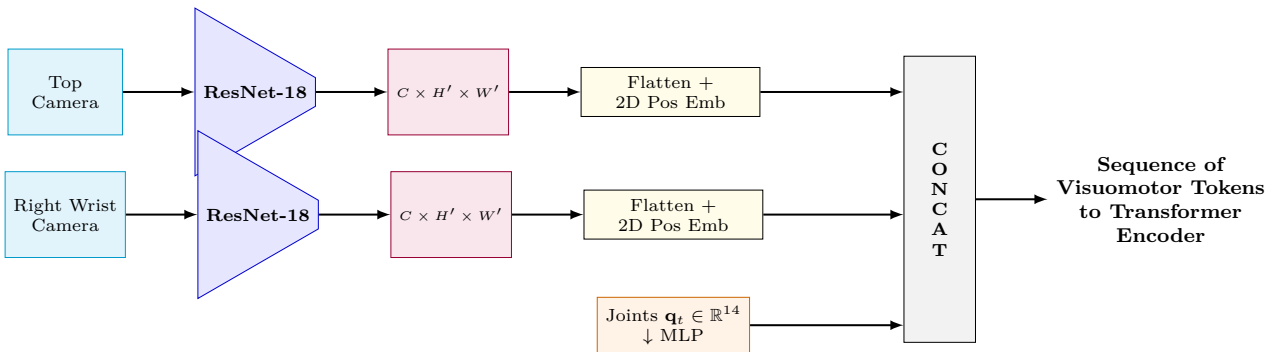


Figure 1: Phase 1: Abstraction and serialization of observation $\mathbf{o}_t$. Images are not compressed into a single vector, but maintain their spatial resolution as hundreds of independent tokens, allowing the Transformer to attend to millimeter-level features across the scene.

2.2 Phase 2: CVAE (Training Only)

While the Transformer Encoder analyzes the visual scene, the system must account for action *multimodality* (the stylistic variance in how an action is executed, given that the same task can be carried out in multiple ways).

This mechanism operates exclusively during training via the **CVAE Encoder (Conditional Variational Autoencoder)** module.

The CVAE Encoder is a compact Transformer that takes the **full Ground Truth trajectory** $\mathbf{A}_t \in \mathbb{R}^{H \times D}$ executed by the human demonstrator as input. Its role is to observe the entire future motion horizon and compress this "intent" into a single low-dimensional vector.

1. **Trajectory Ingestion:** Physically, the ground-truth trajectory $\mathbf{A}_t \in \mathbb{R}^{H \times D}$ is not a spatial curve, but a numerical array: a temporal matrix where each row denotes the exact joint states across all 14 motors at a specific fraction of a second. Over a horizon of $H = 100$, this yields 100 sequential kinematic vectors. This matrix passes through a linear layer and is added to *1D Positional Embeddings*, which act as timestamps indicating the chronological sequence of motor frames.
2. **Injection of the [CLS] Token:** Preceding this sequence of 100 motor vectors, an extra vector (the [CLS] token, short for *Classification*) is prepended. It can be viewed as an uninitialized "scratchpad": a set of learnable parameters initialized without spatial coordinates or temporal bindings, tasked purely with aggregating global information.
3. **Self-Attention and Temporal Aggregation:** The resulting 101 vectors (1 [CLS] + 100 motor steps) pass through the Transformer's Self-Attention layers. Because the [CLS] token has no fixed semantic bias, the attention mechanism forces it to attend to all 100 motor tokens simultaneously. Upon exiting the layers, the [CLS] token has aggregated the holistic context of the trajectory—for instance, encoding the latent concept: *"This is a smooth obstacle avoidance maneuver"*.
4. **Probabilistic Projection:** The network discards the 100 temporal tokens, retaining only the contextualized [CLS] token. This single summary vector is routed through two separate linear layers to parameterize a Gaussian distribution: a mean vector $\boldsymbol{\mu}$ and a variance vector $\boldsymbol{\sigma}^2$.
5. **Sampling and Reparameterization:** Finally, the style vector $\mathbf{z} \in \mathbb{R}^{d_z}$ is sampled from this distribution. To enable gradient backpropagation through a stochastic operation, the *Reparameterization Trick* is applied:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \text{with } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

The resulting vector $\mathbf{z}$ is an ultra-compact representation (e.g., only 32 values) of the original 100×14 trajectory matrix.

Intuitive Example: Resolving Multimodality with the [CLS] Token Imagine the robot must reach for an apple with a bottle placed in the center of the workspace. During data collection, the human operator occasionally maneuvers around the bottle from the **right**, and other times from the **left** (the essence of human behavioral *multiplicity*).

If we pass the trajectory matrix $\mathbf{A}_t$ of a demonstration that routed to the **left** into the CVAE Encoder:

- The matrix contains 100 angular frames that progressively steer the manipulator arm toward the left.
- The [CLS] token attends to all 100 frames and extracts the core motion intent: *"The shoulder joints deviate along the negative X-axis before straightening out"*.
- The token is mapped into $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$, from which $\mathbf{z}$ is sampled.
- Mathematically, $\mathbf{z}$ becomes a 32-dimensional vector occupying a distinct cluster within the latent space (e.g., the left hemisphere of the Gaussian), encoding the decision *"Left-side avoidance style"*.

When this vector $\mathbf{z}$ is subsequently injected into the Transformer Decoder, it functions as a conditioning command: it constrains the Decoder to produce left-avoidance coordinates, preventing it from averaging both modes and colliding into the center bottle.

Crucial Note: At inference, this entire branch is deactivated (since future ground truth $\mathbf{A}_t$ is unavailable). The vector $\mathbf{z}$ is either sampled from a standard Gaussian or set to $\mathbf{0}$ for nominal deterministic execution.

WARNING: Each Gaussian models a specific execution style of a sub-action segment; all Gaussians are regularized by the **KL** loss to cluster near the origin of the latent space, and $\mathbf{z}$ denotes the sampled instance (which exhibits minor variations since it is drawn from a continuous distribution).

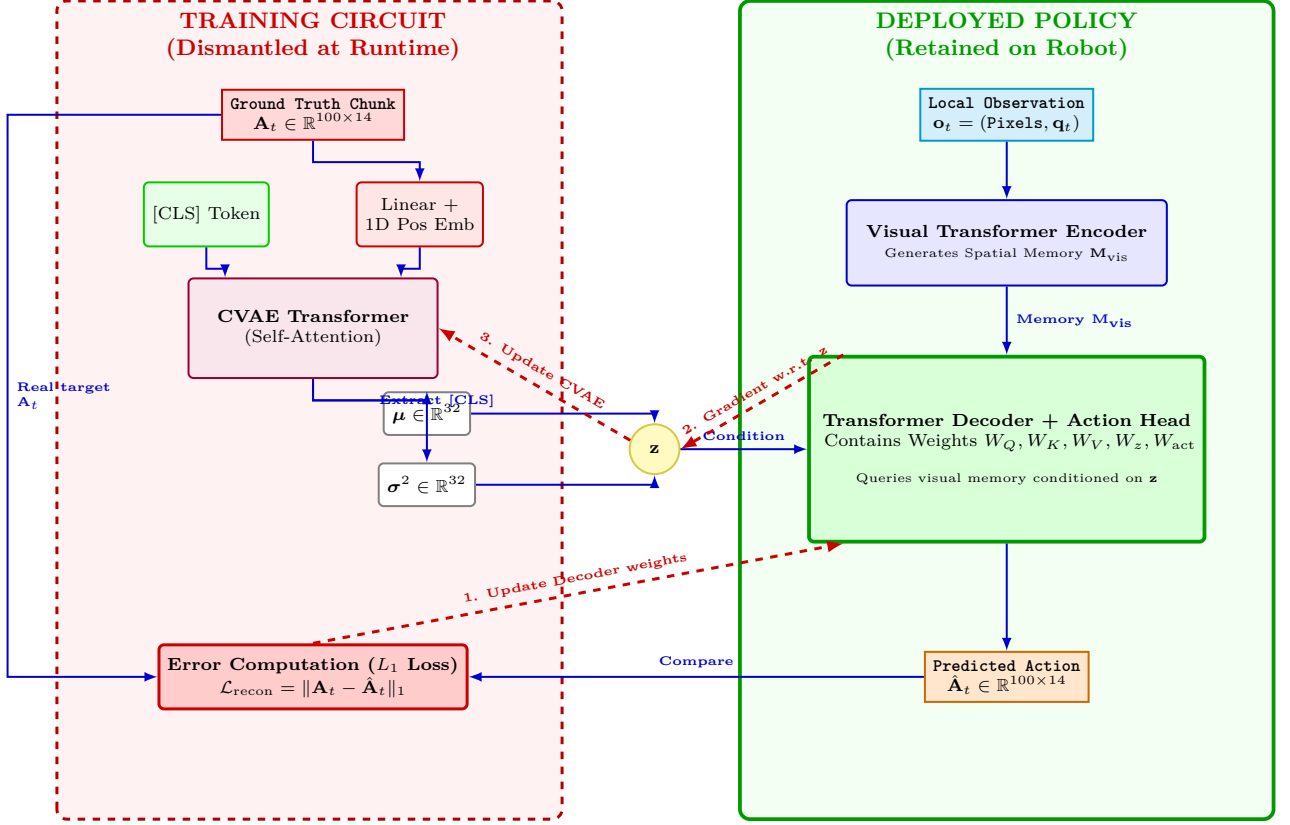


Figure 2: Detailed architecture and co-training cycle in ACT. The left panel (active strictly during training) expands the internal mechanics of the CVAE: the future chunk is aggregated by the [CLS] token and processed by the Transformer to parameterize the Gaussian distribution (μ, σ^2) . From this, the style vector $\mathbf{z}$ is sampled to condition the Decoder (right panel). During backpropagation (dashed red arrows), the L_1 loss updates Decoder weights and flows upstream to calibrate the Encoder. At runtime, the entire left panel is detached and excluded.

2.3 Variational Foundations in ACT: Latent Space, Gaussian Dynamics, and Resolving Multimodality

A fundamental challenge in learning from human teleoperation demonstrations (*Behavioral Cloning*) lies in the **intrinsic multimodality** of the data: given identical visuo-proprioceptive observations $\mathbf{o}_t$, different human operators (or the same operator across trials) can execute radically divergent kinematic trajectories (e.g., maneuvering around an obstacle via the left vs. right path).

When optimizing a conventional deterministic model using standard regression losses (such as L_1 or L_2) directly in action space $\mathbf{A}_t$, expected error minimization forces network parameters to converge toward the *geometric mean* of the demonstrations:

$$\hat{\mathbf{A}}_{\text{collapse}} \approx \mathbb{E}[\mathbf{A}_t | \mathbf{o}_t] = \frac{\mathbf{A}_{\text{left}} + \mathbf{A}_{\text{right}}}{2}$$

In real-world deployment, this averaged trajectory drives the robot end-effector straight into the central obstacle, rendering the policy unusable. To disentangle and isolate these behavioral modes, ACT implements an architecture grounded in a **Conditional Variational Autoencoder (CVAE)**.

2.3.1 Data Structure: Temporal Chunking and Local Distributions

A Gaussian distribution in the CVAE does not model an entire multi-second task episode, but rather a single localized **Action Chunk** (a temporal segment of motion):

$$\mathbf{A}_t = \begin{bmatrix} \mathbf{a}_t \\ \mathbf{a}_{t+1} \\ \vdots \\ \mathbf{a}_{t+H-1} \end{bmatrix} \in \mathbb{R}^{H \times D}$$

Given a teleoperation dataset sampled at a fixed control frequency (e.g., 50 Hz), a sliding temporal window of horizon H (e.g., $H = 100$ steps, corresponding to 2 s) slides along each demonstration episode. A corpus of hundreds of demonstrations produces hundreds of thousands of training pairs $(\mathbf{o}_t, \mathbf{A}_t)$. Each window is mapped into its own local conditional distribution.

2.3.2 The Latent Variable $\mathbf{z}$ and the Necessity of Variance

The CVAE Encoder compresses the local temporal chunk $\mathbf{A}_t$ (solely the next H future steps, typically spanning 1–2 seconds, rather than the global episode) into a low-dimensional latent variable:

$$\mathbf{z} \in \mathbb{R}^{d_z} \quad (\text{typically } d_z = 32)$$

Here, $\mathbf{z}$ represents a continuous coordinate vector formalizing the “style” or specific kinematic execution bounded to this sub-segment (e.g., approach curvature, instantaneous velocity profile, proactive gripper pre-shaping).

Unlike a deterministic autoencoder, the Encoder does not output an isolated discrete vector $\mathbf{z}$, but parameterizes a multivariate Gaussian distribution with a diagonal covariance matrix:

$$\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t = \text{Encoder}_{\text{CVAE}}(\mathbf{A}_t, \mathbf{o}_t)$$

Retaining an explicit standard deviation vector $\boldsymbol{\sigma}_t$ is an indispensable structural design choice:

- **Preventing Sparse Latent Voids (*The Hole Problem*):** If the Encoder projected each chunk into a single deterministic point ($\boldsymbol{\sigma} = \mathbf{0}$), the latent space would consist of isolated singularities scattered across an undefined domain. Any unobserved point queried at test time would trigger out-of-distribution layer activations in the Decoder, causing severe motor jitter.
- **Neighborhood Robustness (*Ball of Tolerance*):** Stochastic sampling via the *Reparameterization Trick*:

$$\mathbf{z} = \boldsymbol{\mu}_t + \boldsymbol{\sigma}_t \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

consistently perturbs the latent representation throughout gradient descent. This forces the Decoder to learn that the entire hyperspherical neighborhood of radius proportional to $\boldsymbol{\sigma}_t$ centered at $\boldsymbol{\mu}_t$ maps consistently to the same coherent trajectory class.

2.3.3 Objective Function: Balancing Reconstruction and KL Divergence

The continuity and coverage of the latent space emerge directly from optimizing the **Evidence Lower Bound** (ELBO) during training:

$$\mathcal{L}_{\text{CVAE}} = \mathcal{L}_{\text{reconstruction}} + \beta \cdot \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{A}_t, \mathbf{o}_t) \parallel p(\mathbf{z}))$$

where the reconstruction loss is computed as the L_1 norm between ground-truth coordinates and Action Head predictions:

$$\mathcal{L}_{\text{reconstruction}} = \frac{1}{H \cdot D} \sum_{i=1}^H \sum_{j=1}^D |a_{t+i-1,j} - \hat{a}_{t+i-1,j}|$$

The regularization term enforces the **Kullback-Leibler Divergence** relative to a standard isotropic Gaussian prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{A}_t, \mathbf{o}_t) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) = -\frac{1}{2} \sum_{k=1}^{d_z} (1 + \log(\sigma_{t,k}^2) - \mu_{t,k}^2 - \sigma_{t,k}^2)$$

This term exerts a crucial centripetal pull:

1. **Centered Regularization** ($\mu_{t,k}^2 \rightarrow 0$): Penalizes wandering centroids, constraining all style clusters to reside within a unit sphere centered at the origin.
2. **Preventing Deterministic Collapse** ($\sigma_{t,k}^2 \rightarrow 1$): Penalizes vanishing variances; as $\sigma_{t,k}^2 \rightarrow 0$, the logarithmic term $\log(\sigma_{t,k}^2) \rightarrow -\infty$, causing the loss to diverge to $+\infty$.

Properly weighting this trade-off via β guarantees that local distributions neither drift apart nor collapse into an uninformative prior (*Posterior Collapse*). The resulting latent manifold forms a **compact, continuous space**: along any linear interpolation path $\mathbf{z}(\alpha) = (1 - \alpha)\mathbf{z}_A + \alpha\mathbf{z}_B$, trajectories decoded by the network transition smoothly and differentially without erratic jumps or singularities.

2.3.4 Structural Decoupling Between Training and Inference

A frequent point of confusion when analyzing ACT is how the robot selects a style vector during deployment (*runtime*). Crucially, **no search, nearest-neighbor query, or Gaussian distance matching occurs at runtime**.

During inference on the real robot, the future trajectory chunk $\mathbf{A}_t$ is inherently unavailable; therefore, the $\text{Encoder}_{\text{CVAE}}$ block is **entirely decoupled and omitted from runtime memory**. The policy operates in one of two modes:

- **Nominal Deterministic Inference ($\mathbf{z} = \mathbf{0}$):** In nearly all closed-loop implementations, the latent vector is fixed directly to zero:

$$\mathbf{z}_{\text{runtime}} = [0, 0, \dots, 0]^T \in \mathbb{R}^{d_z}$$

Because KL regularisation draws all operational modes symmetrically toward the center of the standard prior, the origin $\mathbf{z} = \mathbf{0}$ corresponds to the nominal mode with highest joint density. Removing stochasticity suppresses non-deterministic joint jitter, yielding highly repeatable and smooth motion.

- **Stochastic Exploratory Inference ($\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$):** Should the manipulator experience a grasp failure or kinematic stall, $\mathbf{z}$ can be freshly sampled from the standard Gaussian prior, allowing the policy to instantiate an alternative execution mode.

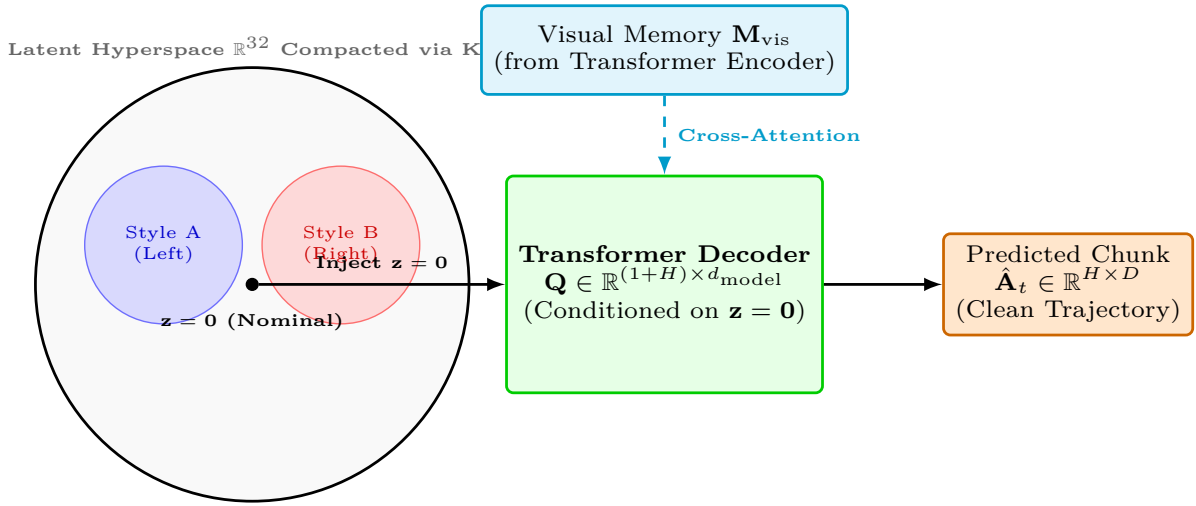


Figure 3: Runtime execution of the CVAE within ACT. The CVAE Encoder is removed. By locking $\mathbf{z} = \mathbf{0}$ at the origin of the regularized latent space, the Transformer Decoder leverages Cross-Attention over perceptual memory $\mathbf{M}_{\text{vis}}$ alone to synthesize the optimal nominal trajectory without collapsing to the mean.

2.4 Phase 3: The Generative Engine (Transformer Encoder-Decoder)

The ACT *Policy* generates actionable kinematic commands $\hat{\mathbf{A}}_t$ by blending the spatial-geometric representation of the workspace (derived from observation $\mathbf{o}_t$) with the modal constraint provided by the latent style vector $\mathbf{z}$.

Unlike causal or autoregressive language models, this generative engine does not predict actions sequentially conditioned on its own prior outputs: it adapts the architecture of **DETR** (*DEtection TRansformer*), framing chunk generation as a parallel prediction problem over a fixed set of query vectors (*Queries*).

2.4.1 The Transformer Encoder (Implicit Spatial Memory Construction)

The Encoder takes the visuomotor token matrix constructed at the current timestep as input:

$$\mathbf{X}_{\text{vis}} = \begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_{N_{\text{tokens}}} \end{bmatrix} \in \mathbb{R}^{N_{\text{tokens}} \times d_{\text{model}}}$$

where each row represents a localized entity (either a 2D patch extracted from the camera feature maps or the scalar joint state $\mathbf{q}_t$, projected to $d_{\text{model}} = 512$ and summed with respective 2D/1D positional embeddings), as detailed earlier.

Within each of the L_{enc} Encoder layers, this matrix is linearly projected by three trainable weight matrices $W_Q^{\text{enc}}, W_K^{\text{enc}}, W_V^{\text{enc}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$Q = \mathbf{X}W_Q^{\text{enc}}, \quad K = \mathbf{X}W_K^{\text{enc}}, \quad V = \mathbf{X}W_V^{\text{enc}}$$

Distributing computation across N_h parallel attention heads ($d_k = d_{\text{model}}/N_h = 512/8 = 64$ dimensions per head), the scalar attention weight between arbitrary tokens i and j is given by:

$$\alpha_{ij} = \frac{\exp\left(\frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d_k}}\right)}{\sum_{l=1}^{N_{\text{tokens}}} \exp\left(\frac{\mathbf{q}_i \mathbf{k}_l^T}{\sqrt{d_k}}\right)}$$

- **Visual Correlation Mechanics:** If token i represents a patch from the overhead camera observing the rim of a mug and token j originates from the wrist camera focused on its handle, training drives down the angular discrepancy between $\mathbf{q}_i$ and $\mathbf{k}_j$. The dot product $\mathbf{q}_i \mathbf{k}_j^T$ increases, yielding a prominent attention weight α_{ij} .
- **Topological Fusion:** The updated representation for token i is calculated as a convex combination:

$$\mathbf{x}'_i = \sum_{j=1}^{N_{\text{tokens}}} \alpha_{ij} \mathbf{v}_j$$

fusing disparate camera viewpoints into a shared, unified reference frame.

Following residual connections, *Layer Normalization*, and a two-layer dense feed-forward network (*FFN*) per layer:

$$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1^{\text{enc}} + \mathbf{b}_1^{\text{enc}}) W_2^{\text{enc}} + \mathbf{b}_2^{\text{enc}}, \quad W_1^{\text{enc}} \in \mathbb{R}^{512 \times 2048}, \quad W_2^{\text{enc}} \in \mathbb{R}^{2048 \times 512}$$

the Encoder outputs the final immutable matrix:

$$\mathbf{M}_{\text{vis}} \in \mathbb{R}^{N_{\text{tokens}} \times d_{\text{model}}}$$

referred to as the **Memory**. This matrix no longer represents isolated raw pixels, but a continuous semantic and volumetric map of the operational environment.

WARNING: As observed across Transformer architectures, the attention module routes and contextualizes features, whereas the **FFN** processes and extracts meaning from them.

2.4.2 The Transformer Decoder (Parallel Trajectory Decoding)

The Decoder queries the perceptual *Memory* $\mathbf{M}_{\text{vis}}$ to output continuous physical coordinates across the entire planning horizon H in a single forward pass. This process proceeds across four distinct stages:

1. Assembling the Query Matrix ($\mathbf{Q}_{\text{in}}$) The Decoder does not receive raw sensor inputs directly; instead, it processes an ordered sequence of latent query tokens:

- **Style Conditioning as Token 0:** The latent vector $\mathbf{z} \in \mathbb{R}^{32}$ (sampled during training, or fixed to $\mathbf{z} = \mathbf{0}_{32}$ at runtime) is projected into the model dimension via linear projection matrix $W_z \in \mathbb{R}^{32 \times 512}$ and bias $\mathbf{b}_z \in \mathbb{R}^{512}$:

$$\mathbf{e}_z = \mathbf{z}W_z + \mathbf{b}_z \in \mathbb{R}^{1 \times 512}$$

- **Temporal Positional Queries:** H temporal query vectors $\mathbf{p}_1, \dots, \mathbf{p}_H \in \mathbb{R}^{1 \times 512}$ are instantiated. These are learnable embeddings independent of the robot’s immediate sensory state; each explicitly encodes an abstract relative timestep along the horizon:

$$\mathbf{p}_h \equiv \text{”Target kinematic configuration for relative step } t + h \cdot \Delta t\text{”}$$

These vectors are stacked sequentially along the sequence dimension ($N_{\text{dec}} = 1 + H$ tokens):

$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{e}_z \\ \mathbf{p}_1 \\ \mathbf{p}_2 \\ \vdots \\ \mathbf{p}_H \end{bmatrix} \in \mathbb{R}^{(1+H) \times 512}$$

2. Decoder Self-Attention (Trajectory-Level Coherence) Within the initial block of each Decoder layer, $\mathbf{Q}_{\text{in}}$ passes through non-causal (bidirectional) Multi-Head Self-Attention:

$$\mathbf{Q}_{\text{self}} = \mathbf{Q}_{\text{in}} W_{\mathbf{Q},\text{self}}^{\text{dec}}, \quad \mathbf{K}_{\text{self}} = \mathbf{Q}_{\text{in}} W_{\mathbf{K},\text{self}}^{\text{dec}}, \quad \mathbf{V}_{\text{self}} = \mathbf{Q}_{\text{in}} W_{\mathbf{V},\text{self}}^{\text{dec}}$$

- Since no lower-triangular causal mask is applied, temporal query $\mathbf{p}_h$ computes attention dot-products with the style token $\mathbf{e}_z$ and all other temporal queries $\mathbf{p}_k$ ($k \neq h$) simultaneously.
- **Style Broadcasting:** Attention between temporal row h and row 0 incorporates the behavioral conditioning of $\mathbf{e}_z$, enforcing consistent stylistic execution across all predicted timesteps.
- **Kinematic Continuity:** Cross-query attention between $\mathbf{p}_h$ and $\mathbf{p}_{h+1}$ regularizes the temporal gradient between successive steps, preventing sudden joint velocity or acceleration spikes.

3. Cross-Attention over Spatial Memory (Querying the Scene) The query matrix produced by Self-Attention, denoted by $\mathbf{U} \in \mathbb{R}^{(1+H) \times 512}$, forms the Queries for Cross-Attention. Keys and Values are projected from perceptual memory $\mathbf{M}_{\text{vis}}$:

$$\mathbf{Q}_{\text{cross}} = \mathbf{U} W_{\mathbf{Q},\text{cross}}^{\text{dec}} \in \mathbb{R}^{(1+H) \times 512}, \quad \mathbf{K}_{\text{cross}} = \mathbf{M}_{\text{vis}} W_{\mathbf{K},\text{cross}}^{\text{dec}} \in \mathbb{R}^{N_{\text{tokens}} \times 512}, \quad \mathbf{V}_{\text{cross}} = \mathbf{M}_{\text{vis}} W_{\mathbf{V},\text{cross}}^{\text{dec}} \in \mathbb{R}^{N_{\text{tokens}} \times 512}$$

Calculating the attention weights governs information extraction:

$$\mathbf{A}_{\text{cross}} = \text{Softmax} \left(\frac{\mathbf{Q}_{\text{cross}} \mathbf{K}_{\text{cross}}^T}{\sqrt{d_k}} \right) \in \mathbb{R}^{(1+H) \times N_{\text{tokens}}}$$

Each row $h \in \{1, \dots, H\}$ represents an attention distribution across all visual and proprioceptive tokens:

- For $h = 1$ ($t + 20$ ms), the query assigns higher attention weights to tokens in $\mathbf{M}_{\text{vis}}$ describing the robot’s current joint state and immediate gripper proximity.
- For $h = H$ (the final horizon step), the query shifts its attention focus toward tokens in $\mathbf{M}_{\text{vis}}$ representing the ultimate goal target (e.g., a placement surface or container).

The product $\mathbf{A}_{\text{cross}} \mathbf{V}_{\text{cross}}$ outputs an updated representation in $\mathbb{R}^{(1+H) \times 512}$, where each future step incorporates the spatial constraints relevant to its execution.

4. Kinematic Extraction and Action Head Following the internal Decoder feed-forward network, row 0 (originating from $\mathbf{z}$) is discarded, having fulfilled its role in conditioning internal layers:

$$\mathbf{H}_{\text{out}} = \mathbf{Y}_{\text{dec}}[1 : H + 1, :] \in \mathbb{R}^{H \times 512}$$

The matrix $\mathbf{H}_{\text{out}}$ passes to the **Action Head**, formulated here as a parallel linear projection across all H rows:

$$\hat{\mathbf{A}}_t = \mathbf{H}_{\text{out}} W_{\text{act}} + \mathbf{b}_{\text{act}}$$

with parameter dimensions:

$$W_{\text{act}} \in \mathbb{R}^{512 \times D}, \quad \mathbf{b}_{\text{act}} \in \mathbb{R}^D$$

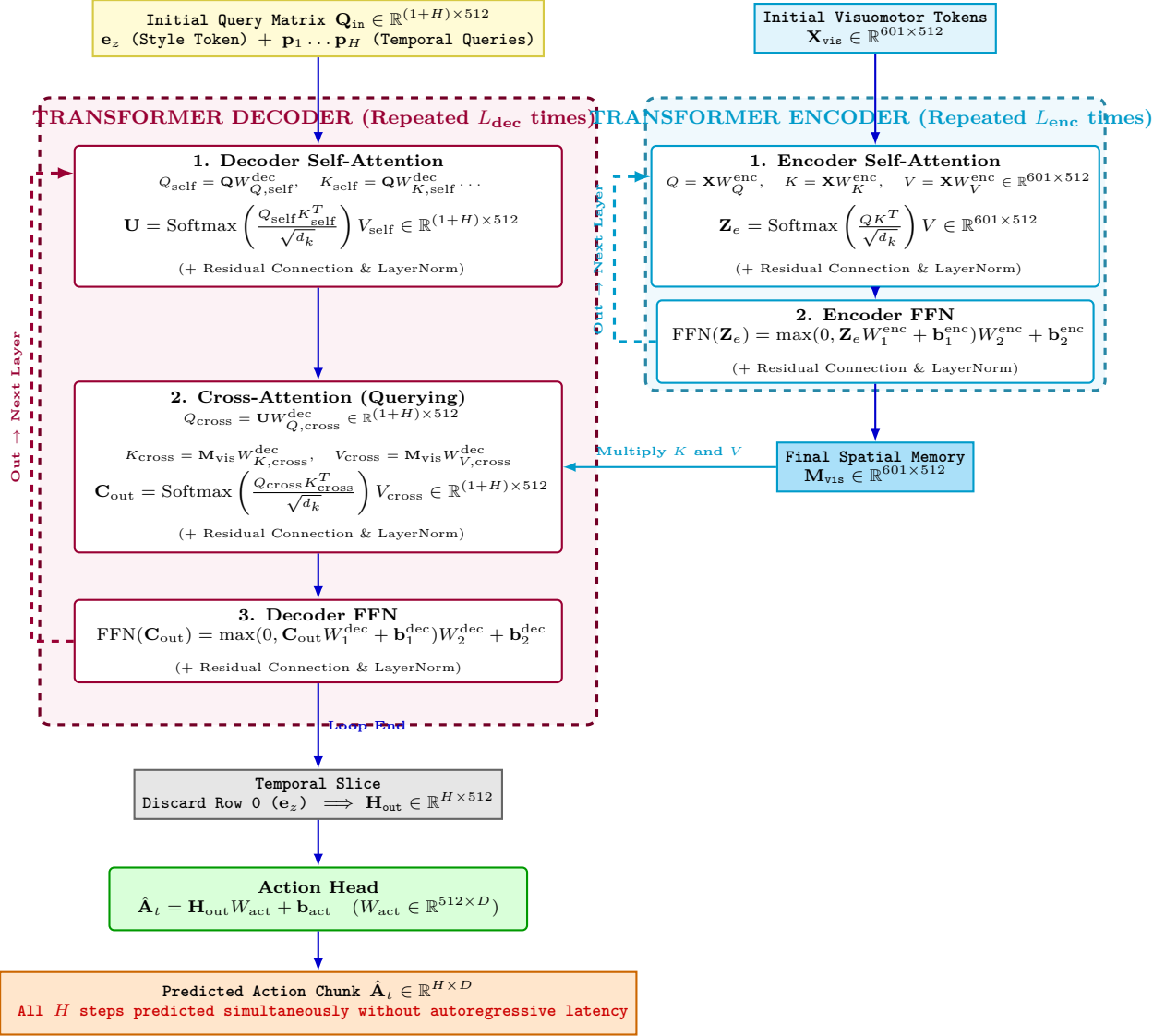


Figure 4: In-depth mathematical breakdown of the Generative Engine. Right: Visual Encoder fuses raw visual tokens via Self-Attention to map geometric correlations across cameras and applies an FFN to extract spatial meaning, yielding Memory $\mathbf{M}_{vis}$. Left: Decoder temporal queries stabilize internally via Self-Attention before projecting Q_{cross} onto visual K_{cross} . The final representations pass through the Action Head to output the entire future kinematic trajectory in parallel.

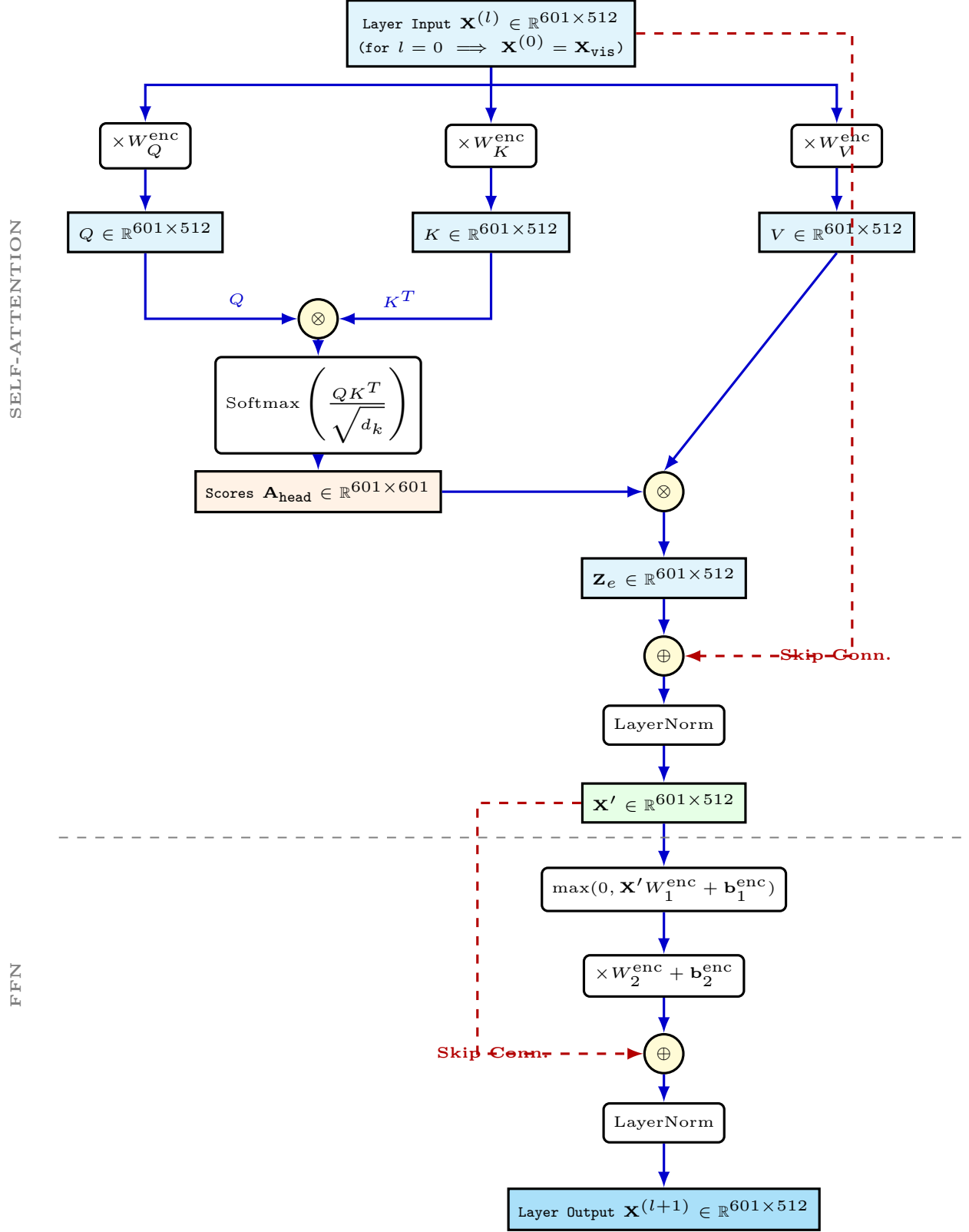


Figure 5: Compact computational graph of a single Transformer Encoder layer. Each step is detailed: linear projections into Query/Key/Value representations via $W_{\{Q,K,V\}}^{\text{enc}}$, matrix multiplication ($\otimes$) to evaluate attention weights $\mathbf{A}_{\text{head}}$, convex aggregation with Values, and non-linear FFN transformation.

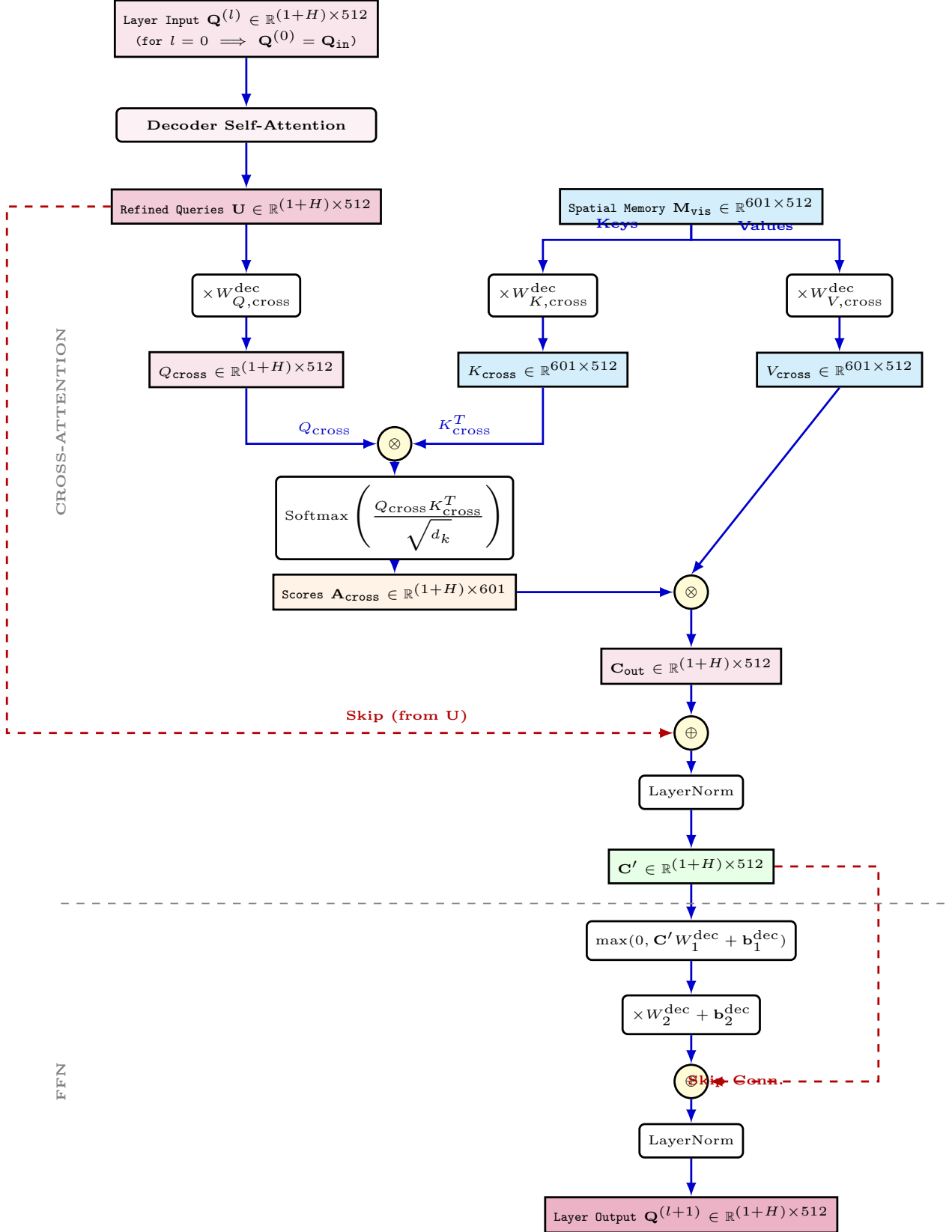


Figure 6: Compact mathematical decomposition of Cross-Attention within the Transformer Decoder. Queries $\mathbf{Q}_{\text{cross}}$ derive from internal temporal sequence $\mathbf{U}$, while Keys $\mathbf{K}_{\text{cross}}$ and Values $\mathbf{V}_{\text{cross}}$ stem from spatial memory $\mathbf{M}_{\text{vis}}$. The normalized dot-product forms attention matrix $\mathbf{A}_{\text{cross}}$, extracting and aggregating relevant visual features into $\mathbf{C}_{\text{out}}$ for each future step.

Rigorous Numerical Walkthrough: Tensor Dimensions and Operations Let us fix standard hyperparameters for a high-performance bimanual cell:

- Temporal prediction horizon: $H = 4$ timesteps (80 ms at 50 Hz).
- Degrees of freedom: $D = 14$ (6 kinematic axes +1 gripper axis per arm).
- Model hidden dimension: $d_{\text{model}} = 512$, split across $N_h = 8$ attention heads ($d_k = 64$).
- Visual scene tokens: $N_{\text{tokens}} = 601$ (3 cameras contributing 200 patches each +1 proprioceptive token $\mathbf{q}_t$).

Let us trace the tensor operations and examine the dimensionality of each weight matrix:

1. **Encoder Spatial Memory:** The input token sequence $\mathbf{X}_{\text{vis}} \in \mathbb{R}^{601 \times 512}$ passes through Encoder Self-Attention. The layer-level transformation:

$$\mathbf{M}_{\text{vis}} = \text{LayerNorm} \left(\text{Softmax} \left(\frac{(\mathbf{X}W_Q^{\text{enc}})(\mathbf{X}W_K^{\text{enc}})^T}{\sqrt{64}} \right) (\mathbf{X}W_V^{\text{enc}}) + \mathbf{X} \right) \in \mathbb{R}^{601 \times 512}$$

stabilizes the internal scene representations. These $601 \times 512 = 307,712$ scalar values capture latent extrinsic geometric constraints, with each row modeling spatial correlations across camera perspectives.

2. **Style Injection and Query Expansion:** At runtime, the controller initializes the deterministic zero vector $\mathbf{z} = [0, 0, \dots, 0]^T \in \mathbb{R}^{1 \times 32}$. Linear projection:

$$\mathbf{e}_z = \mathbf{z}W_z + \mathbf{b}_z = [0, \dots, 0] \begin{bmatrix} w_{1,1} & \dots & w_{1,512} \\ \vdots & \ddots & \vdots \\ w_{32,1} & \dots & w_{32,512} \end{bmatrix} + \mathbf{b}_z = \mathbf{b}_z \in \mathbb{R}^{1 \times 512}$$

extracts bias vector $\mathbf{b}_z$, which was optimized during training to reflect the center of the CVAE-regularized latent space.

The $H = 4$ temporal queries $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4 \in \mathbb{R}^{1 \times 512}$ are concatenated with $\mathbf{e}_z$:

$$\mathbf{Q}_{\text{in}} = \begin{bmatrix} \mathbf{e}_z \\ \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \\ \mathbf{p}_4 \end{bmatrix} = \begin{bmatrix} e_{z,1} & e_{z,2} & \dots & e_{z,512} \\ p_{1,1} & p_{1,2} & \dots & p_{1,512} \\ p_{2,1} & p_{2,2} & \dots & p_{2,512} \\ p_{3,1} & p_{3,2} & \dots & p_{3,512} \\ p_{4,1} & p_{4,2} & \dots & p_{4,512} \end{bmatrix} \in \mathbb{R}^{5 \times 512}$$

3. **Cross-Attention Layer Coupling:** The self-attended queries ($\mathbf{U} \in \mathbb{R}^{5 \times 512}$) and spatial memory ($\mathbf{M}_{\text{vis}} \in \mathbb{R}^{601 \times 512}$) form the Cross-Attention tensors:

$$Q_{\text{cross}} = \mathbf{U}W_{Q,\text{cross}}^{\text{dec}} \in \mathbb{R}^{5 \times 512}, \quad K_{\text{cross}} = \mathbf{M}_{\text{vis}}W_{K,\text{cross}}^{\text{dec}} \in \mathbb{R}^{601 \times 512}$$

The scaled dot product for each head ($d_k = 64$) produces an attention score matrix:

$$\mathbf{S}_{\text{head}} = \frac{Q_{\text{cross}}K_{\text{cross}}^T}{\sqrt{64}} \in \mathbb{R}^{5 \times 601}$$

Applying Softmax row-wise:

$$\mathbf{A}_{\text{head}}[i, j] = \frac{\exp(\mathbf{S}_{\text{head}}[i, j])}{\sum_{k=1}^{601} \exp(\mathbf{S}_{\text{head}}[i, k])}$$

Row $i = 2$ (corresponding to timestep $t + 1$) assigns dominant weight (e.g., $\mathbf{A}[2, 42] = 0.38$) to visual tokens capturing immediate gripper geometry, while row $i = 5$ (corresponding to timestep $t + 4$) focuses attention (e.g., $\mathbf{A}[5, 310] = 0.45$) on tokens identifying the target placement zone.

Multiplication with Values $V_{\text{cross}} \in \mathbb{R}^{601 \times 512}$ aggregates contextualized features:

$$\mathbf{C}_{\text{out}} = \mathbf{A}_{\text{cross}}V_{\text{cross}} \in \mathbb{R}^{5 \times 512}$$

4. **Continuous Action Command Projection:** At the output of the final Decoder layer, the style conditioning token (row 0) is discarded, isolating the refined temporal state vectors:

$$\mathbf{H}_{\text{out}} = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \\ \mathbf{h}_4 \end{bmatrix} \in \mathbb{R}^{4 \times 512}$$

Each vector $\mathbf{h}_i \in \mathbb{R}^{1 \times 512}$ is projected through the linear Action Head ($\mathbf{W}_{\text{act}} \in \mathbb{R}^{512 \times 14}$, $\mathbf{b}_{\text{act}} \in \mathbb{R}^{14}$):

$$\hat{\mathbf{A}}_t = \mathbf{H}_{\text{out}} \mathbf{W}_{\text{act}} + \mathbf{B}_{\text{act}} = \begin{bmatrix} -0.120 & 0.450 & -0.890 & 1.200 & -0.050 & 0.310 & 0.000 & \dots & 0.00 \\ -0.118 & 0.455 & -0.887 & 1.204 & -0.048 & 0.312 & 0.050 & \dots & 0.05 \\ -0.115 & 0.461 & -0.882 & 1.210 & -0.045 & 0.315 & 0.120 & \dots & 0.12 \\ -0.110 & 0.470 & -0.875 & 1.218 & -0.040 & 0.320 & 0.200 & \dots & 0.20 \end{bmatrix} \in \mathbb{R}^{4 \times 14}$$

The first row commands joint angles for $t + 20$ ms, while subsequent rows delineate the trajectory profile out to $t + 80$ ms, bypassing autoregressive decoding latency.

3 Action Chunking and Temporal Ensembling

The key mechanical breakthrough in ACT that mitigates *compounding error* and overcomes the drawbacks of single-step policies is the real-time interaction between Action Chunking and Temporal Ensembling.

3.1 Action Chunking (Mitigating Temporal Myopia)

If a policy predicts actions strictly 1 step at a time, at 50 Hz the delta between successive predictions is merely 20 ms. Stochastic errors in an individual output can abruptly knock the manipulator off course.

By predicting H steps in parallel (e.g., $H = 100$, spanning 2 full seconds into the future), the Transformer enforces **internal kinematic consistency**: the Decoder self-attention layers ensure that the predicted pose at step $h = 50$ smoothly flows into step $h = 51$. High-frequency noise across individual axes is naturally regularized by predicting cohesive chunks.

3.2 Temporal Ensembling (Exponential Blending)

If the robot simply executed each predicted chunk in its entirety ($H = 100$) before processing the next visual frame, it would operate in an open-loop regime for 2 full seconds, failing to react to environmental perturbations (such as an object being bumped).

ACT resolves this via a continuous receding-horizon scheme: 1. At every control step t (e.g., every 20 ms), cameras capture a new observation. 2. The model infers a **new full chunk** of H steps. 3. Consequently, at step t there exist up to H overlapping, concurrent predictions computed across earlier steps.

WARNING: How does the robot select the final command given up to 100 competing predictions for the current instant?

The answer lies in **Temporal Ensembling**. At step t , the setpoint sent to low-level motor controllers $\bar{\mathbf{a}}_t$ is computed as an exponentially weighted moving average across all active predictions targeting that timestep.

The closed-form equation for the action applied at step t is:

$$\bar{\mathbf{a}}_t = \frac{\sum_{i=0}^{H-1} w_i \cdot \hat{\mathbf{a}}_t^{(t-i)}}{\sum_{i=0}^{H-1} w_i}$$

where:

- $\hat{\mathbf{a}}_t^{(t-i)}$ denotes the prediction for timestep t generated earlier at timestep $(t - i)$.
- $w_i = \exp(-m \cdot i)$ is the exponentially decaying weighting factor (governed by decay rate $m \geq 0$).

Physical Impact: Choosing $m > 0$ weights the most recently inferred chunk ($i = 0$) most heavily, keeping the robot responsive to live scene changes. Concurrently, older predictions ($i \rightarrow H$) act as an *inertial smoothing filter*, attenuating high-frequency noise and drastically reducing motor *jerk*.

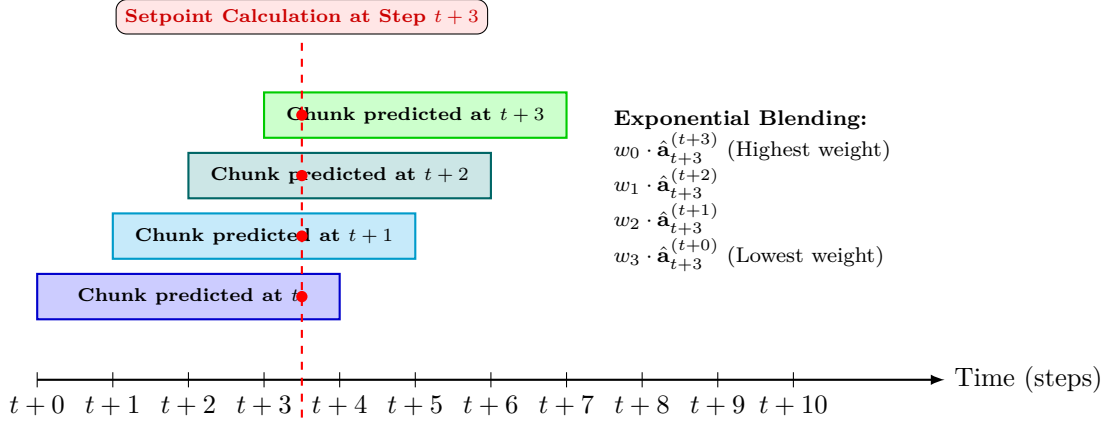


Figure 7: Illustration of *Temporal Ensembling*. At each timestep, the control command is computed by interpolating across overlapping historical chunk predictions targeting that exact step. Exponential weighting w_i ensures kinematic smoothness.

4 Training: Loss Formulation and Gradient Dynamics

The ACT architecture is formulated as a *Conditional Variational Autoencoder* (CVAE). Training optimizes the *Evidence Lower Bound* (ELBO), typically framed in deep learning as minimizing a composite loss function.

The model comprises two distinct sets of learnable parameters:

- ϕ : Parameters of the **CVAE Encoder** (linear projections and latent Transformer Encoder).
- θ : Parameters of the **Policy** (Visual Transformer Encoder, Transformer Decoder, and Action Head).

The overall training objective combines two complementary objectives:

$$\mathcal{L}_{\text{ACT}}(\theta, \phi) = \mathcal{L}_{L1}(\theta, \phi) + \beta \mathcal{L}_{KL}(\phi)$$

where β serves as a regularization weight (following the β -VAE formulation).

4.1 Loss Function Components

1. Reconstruction Loss ($\mathcal{L}_{L1}$): Motor Precision This term penalizes the difference between predicted trajectories $\hat{\mathbf{A}}_t$ and ground-truth human demonstrations $\mathbf{A}_t$. Rather than using Mean Squared Error (L_2), which tends to produce overly smoothed trajectories, ACT employs an L_1 **Loss (Mean Absolute Error)** to maintain sharp, responsive control signals:

$$\mathcal{L}_{L1}(\theta, \phi) = \sum_{h=1}^H \|\hat{\mathbf{a}}_{t+h} - \mathbf{a}_{t+h}\|_1$$

Optimization Impact: The gradient $\nabla \mathcal{L}_{L1}$ propagates backward through the entire architecture. It updates Policy parameters (θ) to improve trajectory decoding, while the *Reparameterization Trick* ($\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$) allows gradients to flow across $\mathbf{z}$ back into the CVAE Encoder (ϕ). This conditions the CVAE to encode style representations that effectively assist the Decoder in minimizing trajectory error.

2. Kullback-Leibler Divergence ($\mathcal{L}_{KL}$): Latent Space Regularization Without constraints, the CVAE Encoder could scatter trajectory encodings arbitrarily throughout the latent space, hindering style sampling at test time. The KL Divergence pulls the learned distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ toward an uninformative standard Gaussian prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$:

$$\mathcal{L}_{KL}(\phi) = D_{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) = \frac{1}{2} \sum_{i=1}^{32} (\sigma_i^2 + \mu_i^2 - 1 - \ln(\sigma_i^2))$$

Optimization Impact: This loss depends strictly on CVAE distribution parameters. Its gradient ($\nabla \mathcal{L}_{KL}$) updates **only** Encoder parameters ϕ , regularizing means μ_i toward 0 and variances σ_i^2 toward 1. Carefully tuning β prevents *posterior collapse*, preserving meaningful modal representations in $\mathbf{z}$.

4.2 The Causal Paradox: Why Discard the CVAE at Runtime?

During training, the model operates as an Autoencoder: it encodes the future trajectory ($\mathbf{A}_t$) into $\mathbf{z}$ and reconstructs it as $\hat{\mathbf{A}}_t$. At test time (inference), the robot operates strictly in the present: **the future chunk $\mathbf{A}_t$ does not yet exist**, as it is precisely what the robot must predict.

Consequently, the **CVAE Encoder cannot be evaluated at runtime** and must be detached.

Latent Footprint on the Deployed Policy During training, the architecture functions as an autoencoder: it inspects future actions $\mathbf{A}_t$ via the CVAE, compresses them into $\mathbf{z}$, and decodes $\hat{\mathbf{A}}_t$ conditioned on this privileged signal. At deployment, however, future actions $\mathbf{A}_t$ are unavailable: the policy must synthesize them using only current observations.

Consequently, input to the CVAE Encoder is missing at runtime, making this module physically unusable. Removing the CVAE structure is therefore a strict causal requirement. How, then, does the offline CVAE benefit the runtime policy? The answer lies in how it shapes the **Transformer Decoder weights (θ)**.

The CVAE as a Modality Disentanglement Mechanism Consider a multimodal scenario (a behavioral "bifurcation"): when bypassing an obstacle, human demonstrators may navigate either to the right or to the left. Without style conditioning, the Decoder would see identical images paired with opposing action outputs. Minimizing L_1 loss would force the network to average both options, causing the robot to collide straight into the center obstacle (*Mode Collapse*).

The CVAE breaks this symmetry. During training, it supplies the Decoder with distinct latent vectors $\mathbf{z}$ for left-avoidance versus right-avoidance trajectories. Rather than learning an averaged motion, the Decoder **learns a conditioned policy**: it carves distinct neural pathways into its feed-forward weights and attention projections, separating different behavioral modes.

Why the Policy Responds Dynamically to $\mathbf{z} = \mathbf{0}$ At runtime, $\mathbf{z}$ is deterministically set to the zero vector ($\mathbf{z} = [0, 0, \dots, 0]$), corresponding to the center of the Gaussian prior. While this input is constant, for a Decoder trained with CVAE conditioning, $\mathbf{z} = \mathbf{0}$ acts as a **"Nominal Mode Trigger"**.

When $\mathbf{z} = \mathbf{0}$ is provided, it activates a learned linear projection (via matrix W_z) that conditions the Transformer into a balanced configuration representing successful nominal human demonstrations. **From this point forward, specific trajectory selection is guided by Visual Cross-Attention:**

- If visual features show an obstacle on the right, Cross-Attention queries the visual memory $\mathbf{M}_{\text{vis}}$ accordingly, routing activations through pathways that steer the arm left.
- If the obstacle appears on the left, the same network layers, attending to the updated visual context, direct the trajectory to the right.

In summary, $\mathbf{z}$ provides the training scaffolding required for the Decoder to learn distinct, non-collapsing trajectory modes. Setting it to zero at deployment yields a nominal trajectory that preserves multimodal stability and reacts dynamically to live visual observations.

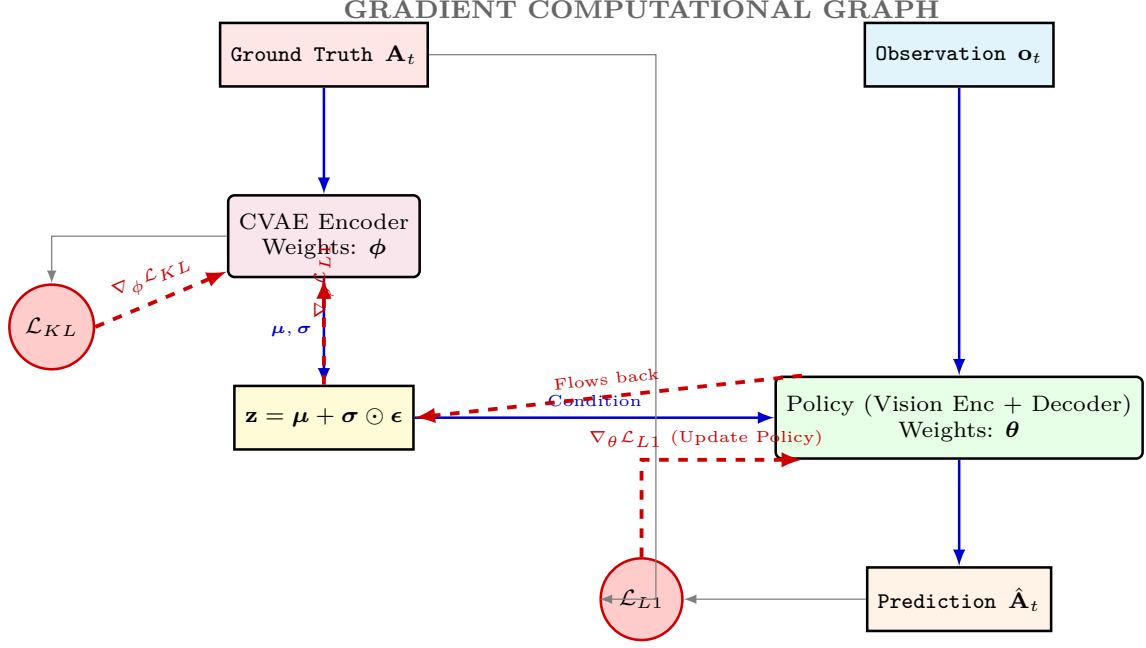


Figure 8: Flow of *Backpropagation*. The **Policy** (θ) is trained strictly via $\mathcal{L}_{L1}$ to minimize behavioral imitation error. In contrast, the **CVAE** (ϕ) receives gradients from two sources: from $\mathcal{L}_{KL}$ (pulling latent distributions toward the prior) and from $\mathcal{L}_{L1}$ (flowing through $\mathbf{z}$ via the *reparameterization trick*, forcing the CVAE to provide representations that directly aid trajectory reconstruction).